FORMAL LANGUAGES / PROOF DEVELOPMENT

Parikh's Theorem

A constructive proof for formalization

A systematic and pedagogical exposition

Prepared for Umang Mathur

11 October 2026

The result. The Parikh image of every context-free language is a finite union of linear sets. Conversely, every semilinear set is the Parikh image of a regular language.

The method. Cut a derivation tree into a small complete tree and bounded one-hole contexts. Record which contexts occur at least once. Prove that, once these contexts have been introduced, their multiplicities can vary independently.

The intended use. This is a mathematical blueprint for mechanization. Definitions, induction measures, compatibility conditions, and both inclusions of the final set equality are explicit. The document contains no claimed machine-checked proof or dependency on a particular proof-assistant library.

The proof works with arbitrary finite context-free grammars. Empty right-hand sides, unit productions, unproductive symbols, and ambiguous derivations require no preprocessing. All natural numbers include zero.

Contents

1 The statement and the proof contract **2**
1.1 Words become vectors . . . 2
1.2 Linear and semilinear sets . . . 2
1.3 What the proof will establish . . . 3

2 A map of the argument **3**

3 Grammars and derivation trees **4**
3.1 The grammar model . . . 4
3.2 A representation that handles empty productions correctly . . . 4
3.3 Occurrences are positions, not just labels . . . 5

4 Contexts and the local surgery lemmas **6**
4.1 One distinguished hole . . . 6
4.2 Inserting a cycle into a complete tree . . . 7

5 Finite building blocks and exact decomposition **8**
5.1 Simple trees . . . 8
5.2 The bounded cut . . . 8
5.3 Decomposing every complete tree . . . 9

6 Which sets of contexts can be used together? **10**
6.1 An explicit compatibility condition . . . 10
6.2 Why first occurrences suffice . . . 11
6.3 After activation, multiplicities are independent . . . 11

7 The exact semilinear representation **12**
7.1 What is and is not canonical . . . 13

8 A worked example: activation has a cost **13**

9 The converse: a regular language for every semilinear set **14**

10 Effectivity and an explicit grammar-size bound **15**
10.1 Deciding activation by a finite procedure . . . 15
10.2 The algorithm producing the representation . . . 16
10.3 A fully explicit, deliberately coarse bound . . . 16
10.4 Edge cases that the proof already covers . . . 18

11 A blueprint for mechanization **18**
11.1 Start with witness data, not just an existential theorem . . . 19
11.2 Represent validity explicitly . . . 19
11.3 The modules and their contracts . . . 20
11.4 Induction measures and finite choices . . . 20
11.5 The finite-support lemma deserves its own interface . . . 21
11.6 A recommended order of milestones . . . 21
11.7 Common gaps to guard against . . . 21

A Derivation trees and ordinary word derivations **22**
A.1 From a complete tree to a word derivation . . . 22
A.2 From a word derivation to a complete tree . . . 22

B An immediate arithmetic corollary **23**

C Notation and the final witness at a glance **24**

Sources and relation to other proofs **24**

1 The statement and the proof contract

1.1 Words become vectors

Fix a finite terminal alphabet Σ. We use $\mathbb{N}^\Sigma$ for the set of functions from Σ to $\mathbb{N}$, with pointwise addition. This avoids choosing an order on the alphabet. If $\Sigma = \{a_1, \ldots, a_d\}$ has been ordered, this is simply $\mathbb{N}^d$.

Definition 1.1 (Parikh map)**.** For a word $w \in \Sigma^*$, its Parikh vector $\Psi(w) \in \mathbb{N}^\Sigma$ is defined by

$$\Psi(w)(a) = \text{the number of occurrences of } a \text{ in } w.$$

For a language $L \subseteq \Sigma^*$, let $\Psi(L) = \{\Psi(w) : w \in L\}$.

For example, over the ordered alphabet (a, b, c),

$$\Psi(\texttt{abacb}) = (2, 2, 1) = \Psi(\texttt{bbaac}).$$

The two words differ, but their counts agree. Write e_a for the vector with value 1 at a and 0 elsewhere. Directly from counting,

$$\Psi(\varepsilon) = 0, \qquad \Psi(a) = e_a, \qquad \Psi(uv) = \Psi(u) + \Psi(v). \tag{1.1}$$

For the last identity, fix a letter a. Its occurrences in uv split into the occurrences in u and those in v. Equality at every coordinate gives equality of vectors. Induction then gives additivity for any finite concatenation and $\Psi(w^k) = k\Psi(w)$.

1.2 Linear and semilinear sets

Definition 1.2 (Linear set)**.** For a base vector $b \in \mathbb{N}^\Sigma$ and a finite indexed family of period vectors $(p_i)_{i \in I}$, define

$$\mathrm{Lin}(b; (p_i)_{i \in I}) = \left\{ b + \sum_{i \in I} k_i p_i : (k_i)_{i \in I} \in \mathbb{N}^I \right\}.$$

A set is *linear* if it has this form, and *semilinear* if it is a finite union of linear sets.

The empty index set gives $\mathrm{Lin}(b; ()) = \{b\}$. The empty union gives $\varnothing$. A period may be zero, and different indices may carry the same period vector. These conventions are useful for grammars with silent cycles.

Example 1.3. The set $\{(n, n) : n \in \mathbb{N}\}$ is linear, with base $(0, 0)$ and period $(1, 1)$. The set

$$\{(2 + 3m, n) : m, n \in \mathbb{N}\} \ \cup \ \{(0, 0)\}$$

is semilinear: its first component has base $(2, 0)$ and periods $(3, 0), (0, 1)$; its second component is a singleton.

Theorem 1.4 (Parikh's theorem, with its converse)**.** *For every finite context-free grammar G over Σ, the set $\Psi(L(G))$ is semilinear. Moreover, the proof gives an effective procedure to obtain a finite semilinear representation from G, with explicit grammar-dependent bounds on the number of components, the number of periods, and all vector magnitudes (theorem 10.3).*

Conversely, for every semilinear set $X \subseteq \mathbb{N}^\Sigma$, one can effectively construct a regular language $R \subseteq \Sigma^*$ such that $\Psi(R) = X$.

Thus every context-free language has the same Parikh image as some regular language. This statement does *not* say that the original language is regular, or that every language with that Parikh image is context-free.

For instance, $\{a^n b^n c^n : n \in \mathbb{N}\}$ and $(abc)^*$ have the same Parikh image, although the former is not context-free. This familiar example is only an illustration; its non-context-freeness is not used in the proof.

1.3 What the proof will establish

We will define a finite set $\mathcal{B}$ of complete trees and a finite set $\mathcal{C}$ of repeatable contexts. For each $B \in \mathcal{B}$ and each suitable subset $D \subseteq \mathcal{C}$, the proof constructs a linear set

$$\left\{ \operatorname{wt}(B) + \sum_{C \in D} \operatorname{wt}(C) + \sum_{C \in D} k_C \operatorname{wt}(C) : k_C \in \mathbb{N} \right\}. \tag{1.2}$$

The first sum pays for *one occurrence of every chosen context*. The second sum accounts for additional occurrences. We will prove that the union of precisely these components is $\Psi(L(G))$.

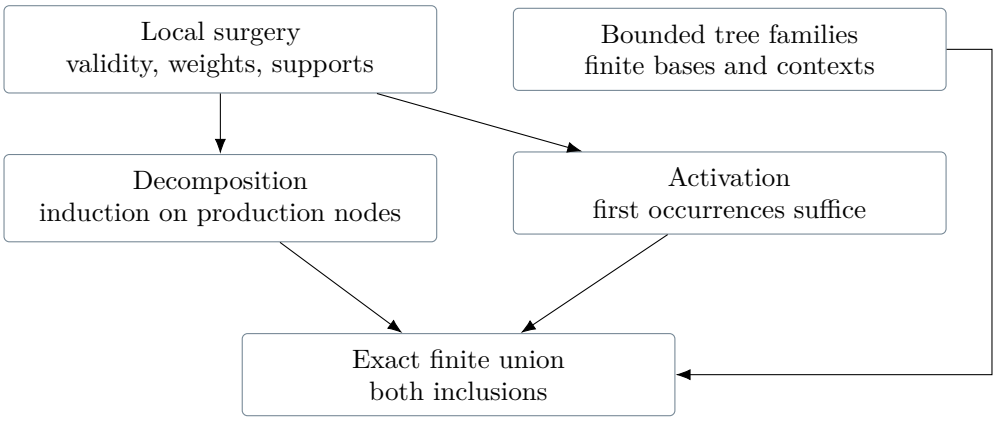
The word "suitable" matters. A context rooted at A can only be inserted into a tree that contains an A-node. One context may introduce the nonterminal needed by another. Establishing this compatibility condition is the central combinatorial issue after tree decomposition.

2 A map of the argument

A first picture. For the grammar $S \to aSb \mid \varepsilon$, start with the tree using $S \to \varepsilon$. Surround its S-root with the context for $S \to a\Box_S b$. Each such insertion adds one a and one b. After r insertions the yield is $a^r b^r$ and the count vector is $r(1,1)$. A small tree plus repeatable additions has produced a linear set. For a general grammar there are several kinds of additions, and we must also determine which kinds can coexist.

The argument has five stages. Each stage isolates a different proof obligation.

1. **Local surgery.** A context with an A-root and one A-hole can surround an existing A-subtree. The result is still a valid derivation tree. Terminal counts add, and the set of nonterminals present can only grow.
2. **Bounded decomposition.** Repeatedly remove a bounded context from a non-simple tree. The number of production nodes strictly decreases, so the process terminates at a simple complete tree.
3. **Activation.** From a valid insertion sequence, retain contexts in order of first occurrence. Each retained context is still enabled. After this first pass, all retained contexts can be inserted arbitrarily often.
4. **Finite union.** There are finitely many simple complete trees, finitely many bounded contexts, and finitely many subsets of those contexts. Each compatible pair gives exactly one linear component.
5. **Converse.** Realize a base vector by one word and each period vector by another word. A finite union of expressions of the form $uv_1^* \cdots v_k^*$ has the required Parikh image.



Reading order. Sections 3–7 contain the complete forward proof. Section 8 is a worked example of the subtle activation step. Section 9 proves the converse. Sections 10 and 11 explain effective construction and mechanization. The appendices supply the connection to ordinary word derivations and a small arithmetic corollary.

Dependencies. The forward proof uses induction on finite trees and natural numbers, finite enumeration, and elementary arithmetic in $\mathbb{N}^{\Sigma}$. It does not invoke a grammar normal form, the pumping lemma, Dickson's lemma, a theorem about Diophantine equations, or Presburger quantifier elimination.

3 Grammars and derivation trees

3.1 The grammar model

Fix a grammar $G = (N, \Sigma, P, S)$, where N and Σ are disjoint finite sets, $S \in N$, and P is a finite set of productions

$$A \to X_1 \cdots X_m, \qquad A \in N, \quad X_i \in N \uplus \Sigma, \quad m \geq 0.$$

The disjoint union notation records whether a symbol is a nonterminal or a terminal. Let $n = |N|$. Since $S \in N$, we have $n \geq 1$.

A sentential form is a word over $N \uplus \Sigma$. The relation $\alpha \Rightarrow_G \beta$ means that one occurrence of some A in α is replaced by the right-hand side of a production for A. Its reflexive transitive closure is $\Rightarrow_G^*$. The generated language is

$$L(G) = \{w \in \Sigma^* : S \Rightarrow_G^* w\}.$$

There are no restrictions on right-hand sides. In particular, $A \to \varepsilon$ and $A \to B$ are permitted.

3.2 A representation that handles empty productions correctly

Definition 3.1 (Complete grammar tree)**.** A complete grammar tree is built using the following constructors.

- For each $a \in \Sigma$, $\mathrm{leaf}(a)$ is a tree with root symbol a.
- If $p = (A \to X_1 \cdots X_m) \in P$ and $T_1, \ldots, T_m$ are complete trees with root symbols $X_1, \ldots, X_m$, then
$$\mathrm{node}(p; T_1, \ldots, T_m)$$
is a complete tree with root symbol A.

Write $\mathcal{T}_A$ for the complete trees with root symbol $A \in N$.

Children are ordered. A production node carries its production, not merely its left-hand nonterminal. This records the local validity evidence explicitly. A node for $A \to \varepsilon$ has no children and is a *production node*, even though it is a leaf of the underlying graph. It is not an unexpanded A.

Definition 3.2 (Yield, weight, size, height, and support)**.** For complete trees, define

$$\begin{aligned}
\operatorname{yield}(\operatorname{leaf}(a)) &= a, & \operatorname{yield}(\operatorname{node}(p; T_1, \dots, T_m)) &= \operatorname{yield}(T_1) \cdots \operatorname{yield}(T_m),\\
\operatorname{wt}(T) &= \Psi(\operatorname{yield}(T)), & & \\
\operatorname{size}(\operatorname{leaf}(a)) &= 0, & \operatorname{size}(\operatorname{node}(p; T_1, \dots, T_m)) &= 1 + \sum_{i=1}^{m} \operatorname{size}(T_i),\\
\operatorname{ht}(\operatorname{leaf}(a)) &= 0, & \operatorname{ht}(\operatorname{node}(p; T_1, \dots, T_m)) &= 1 + \max_i \operatorname{ht}(T_i),\\
\operatorname{supp}(\operatorname{leaf}(a)) &= \varnothing, & \operatorname{supp}(\operatorname{node}(p; T_1, \dots, T_m)) &= \{A\} \cup \bigcup_i \operatorname{supp}(T_i),
\end{aligned}$$

where A is the left-hand side of p and the maximum of an empty family is 0.

Thus size counts production nodes, height counts production nodes along a longest path, and support records which nonterminals label production nodes. For $p = (A \to \varepsilon)$, the tree $\operatorname{node}(p;)$ has yield ε, weight 0, size 1, height 1, and support $\{A\}$.

Proposition 3.3 (Trees and word derivations)**.** *For every $A \in N$ and $w \in \Sigma^*$,*

$$A \Rightarrow_G^* w \quad \Longleftrightarrow \quad \exists T \in \mathcal{T}_A \;\; \operatorname{yield}(T) = w.$$

Consequently $\Psi(L(G)) = \{\operatorname{wt}(T) : T \in \mathcal{T}_S\}$.

Proof. A complete proof, using partial trees to track a word derivation one step at a time, is given in Appendix A. The forward proof below uses only the displayed consequence. □

3.3 Occurrences are positions, not just labels

A position in an ordered tree is a finite sequence of child indices. The empty sequence ε names the root. If q names a node with m children, then qi names its ith child, for $1 \le i \le m$. Write $T|_q$ for the subtree at q.

If q names a production node, replacing $T|_q$ by any complete tree with the same root symbol preserves local validity: the parent still sees the same child-root symbol, and all other productions are unchanged. At the root there is no parent, and the root symbol is preserved directly.

> **Key point.** An insertion is made at a particular occurrence of a nonterminal. The support set tells us that a suitable occurrence exists; the position tells us where to perform the insertion.

4 Contexts and the local surgery lemmas

4.1 One distinguished hole

Extend the tree syntax with a constructor $\Box_A$ for each $A \in N$. Its root symbol is A. It is not a production node. In a production node, a hole may occupy a child position whose required root symbol is its label, and holes may also occur deeper in its children.

An *open grammar tree* is a tree in this extended syntax satisfying the same production and child-root conditions. A *one-hole context* is an open grammar tree containing exactly one hole. Complete trees contain no holes. If E has hole $\Box_A$ and $T \in \mathcal{T}_A$, let $E[T]$ be the tree obtained by replacing that hole by T.

For an open tree, define weight by summing e_a over all terminal leaves. Define size, height, and support by the same recursions as before, with

$$\mathrm{wt}(\Box_A) = 0, \qquad \mathrm{size}(\Box_A) = 0, \qquad \mathrm{ht}(\Box_A) = 0, \qquad \mathrm{supp}(\Box_A) = \varnothing.$$

In particular, *the hole does not contribute its label to support.* This convention makes support under plugging exactly additive by union.

Definition 4.1 (Cycle context)**.** A cycle context is a valid one-hole context C whose root symbol and hole label are the same nonterminal A, and which is not the one-node context $\Box_A$. Write $\mathrm{root}(C) = A$.

The word "cycle" refers to returning from A to another occurrence of A along the path to the hole. The tree itself is finite and acyclic. A cycle context contains at least one production node, so

$$\mathrm{size}(C) \geq 1, \qquad \mathrm{root}(C) \in \mathrm{supp}(C). \tag{4.1}$$

Its terminal frontier has the form $u\Box_A v$, for terminal words u, v. Consequently $\mathrm{wt}(C) = \Psi(u) + \Psi(v)$. The hole is a placeholder; its label is not counted as a terminal.

Lemma 4.2 (Plugging identities)**.** *Let E be a valid one-hole context with hole $\Box_A$, and let $T \in \mathcal{T}_A$. Then $E[T]$ is complete and valid, has the root symbol of E, and satisfies*

$$\mathrm{wt}(E[T]) = \mathrm{wt}(E) + \mathrm{wt}(T), \tag{4.2}$$
$$\mathrm{size}(E[T]) = \mathrm{size}(E) + \mathrm{size}(T), \tag{4.3}$$
$$\mathrm{supp}(E[T]) = \mathrm{supp}(E) \cup \mathrm{supp}(T). \tag{4.4}$$

Proof. Induct on the number of edges from the root of E to its unique hole.

If that number is zero, then $E = \Box_A$ and $E[T] = T$. All claims follow from the definitions for a hole.

Otherwise the root is a production node. Exactly one child, say the jth, contains the hole. The other children are complete. Apply the induction hypothesis to the jth child and leave all other children unchanged. The replaced child retains its root symbol, so the production at the root remains valid. There is no hole after replacement.

At a production node, weight is the sum of child weights, size is one plus the sum of child sizes, and support is the union of the root label and child supports. Substituting the three induction identities into these recursions proves all three displayed equations. The root production is unchanged, hence so is its root symbol. □

4.2 Inserting a cycle into a complete tree

Definition 4.3 (Insertion)**.** For complete nonterminal-rooted trees T, T' and a cycle context C with root A, write $T \xrightarrow{C} T'$ if there is a production-node position q of T with root label A such that T' is obtained by replacing $T|_q$ with $C[T|_q]$.

The old subtree is retained inside the new context. We insert structure around it; we do not discard it.

Lemma 4.4 (Insertion is enabled exactly by support)**.** *For a complete nonterminal-rooted tree T and a cycle context C, there exists T' with $T \xrightarrow{C} T'$ if and only if* $\mathrm{root}(C) \in \mathrm{supp}(T)$.

Proof. An insertion supplies a production-node occurrence of $\mathrm{root}(C)$, so the label lies in support. Conversely, membership in support supplies such a position q. Plug $T|_q$ into C and replace the subtree at q. The plugging lemma and equality of root labels give a valid complete tree. □

Lemma 4.5 (Insertion preserves the old tree's resources)**.** *If $T \xrightarrow{C} T'$, then T' has the same root symbol as T, and*

$$\mathrm{wt}(T') = \mathrm{wt}(T) + \mathrm{wt}(C), \tag{4.5}$$
$$\mathrm{size}(T') = \mathrm{size}(T) + \mathrm{size}(C), \tag{4.6}$$
$$\mathrm{supp}(T') = \mathrm{supp}(T) \cup \mathrm{supp}(C). \tag{4.7}$$

Proof. Let $U = T|_q$ be the selected subtree and let E be the outer context obtained from T by replacing U by a hole of the same root label. Then

$$T = E[U], \qquad T' = E[C[U]].$$

Apply lemma 4.2 first to $C[U]$ and then to the outer plugging by E. For weight and size, associate and commute addition. For support, associate and commute union. The outer context gives the same root symbol in both trees. □

Corollary 4.6 (Accounting along an insertion sequence)**.** *If*

$$T_0 \xrightarrow{C_1} T_1 \xrightarrow{C_2} \cdots \xrightarrow{C_m} T_m,$$

then

$$\mathrm{wt}(T_m) = \mathrm{wt}(T_0) + \sum_{i=1}^{m} \mathrm{wt}(C_i),$$
$$\mathrm{supp}(T_m) = \mathrm{supp}(T_0) \cup \bigcup_{i=1}^{m} \mathrm{supp}(C_i).$$

Every intermediate tree is complete and has the root symbol of T_0.

Proof. Induct on m and apply lemma 4.5 at the last step. For $m = 0$, the sum and union over contexts are empty. □

> **Key point.** Future insertions depend only on the presence of nonterminal labels. Insertion never removes a label that was already present. This monotonicity will let us discard repeated uses while deciding which contexts can be activated.

5 Finite building blocks and exact decomposition

5.1 Simple trees

Definition 5.1 (Simple complete tree)**.** A complete tree is simple if no root-to-leaf path contains two production nodes with the same nonterminal label.

Repeated labels in different branches are permitted. For example, a tree for $S \to AA$, with each $A \to a$, is simple when $S \neq A$. Simplicity is a condition on each path, not on the total number of occurrences of a label in the whole tree.

Lemma 5.2 (Height of a simple tree)**.** *Every simple complete tree has height at most $n = |N|$.*

Proof. Along any path, the production-node labels are pairwise distinct members of the n-element set N. The path therefore contains at most n production nodes. Taking the maximum over paths gives the height bound. A terminal-only tree has height zero and also satisfies it. □

Definition 5.3 (The two finite families)**.** Let

$$\mathcal{B} = \{B \in \mathcal{T}_S : B \text{ is simple}\},$$
$$\mathcal{C} = \{C : C \text{ is a cycle context and } \operatorname{ht}(C) \leq n + 1\}.$$

We deliberately put *all* bounded valid cycle contexts in $\mathcal{C}$. We do not require that a context has already appeared in some successful derivation from S. Such a requirement would complicate effective enumeration. The activation condition will reject contexts that cannot be used with a particular base.

Lemma 5.4 (Finiteness and effective enumeration)**.** *For every $H \in \mathbb{N}$, there are finitely many valid open grammar trees of height at most H, and they can be effectively enumerated. Consequently $\mathcal{B}$ and $\mathcal{C}$ are finite and effectively enumerable.*

Proof. At height zero, the only possibilities are terminal leaves and holes. There are finitely many of both because Σ and N are finite.

Suppose the family at height at most H is finite and enumerable. A tree of height at most $H + 1$ is either a terminal leaf, a hole, or a production node $\operatorname{node}(p; T_1, \ldots, T_m)$. There are finitely many choices for $p \in P$. For a fixed p, its arity m is fixed and finite. Each child comes from the enumerated family at height at most H. Enumerate the finite Cartesian product of these child families, and retain tuples whose root symbols match the right-hand side of p. This generates precisely the valid trees at the new bound.

For $\mathcal{B}$, enumerate to height n and filter for no holes, root S, and simplicity. The height lemma shows that every simple S-tree is included. For $\mathcal{C}$, enumerate to height $n + 1$ and filter for exactly one hole, matching root and hole labels, and nontriviality. All these tests concern finite trees and are decidable. □

5.2 The bounded cut

An arbitrary repetition in a large tree may enclose very large side branches. Cutting there would not immediately give a bounded context. We instead locate a non-simple subtree whose proper subtrees are all simple.

Lemma 5.5 (A non-simple tree contains a bounded removable context)**.** *Let $T \in \mathcal{T}_A$ be non-simple. Then there exist $T' \in \mathcal{T}_A$ and $C \in \mathcal{C}$ such that*

$$T' \xrightarrow{C} T \qquad \text{and} \qquad \operatorname{size}(T') < \operatorname{size}(T).$$

The insertion reconstructs T exactly, including its production labels and child order.

Proof. **1. Choose a minimal non-simple subtree.** Among the non-simple subtrees of T, choose one, U, with the least number of production nodes. This is a nonempty finite collection because T itself is non-simple. Every proper nonterminal-rooted subtree of U is smaller than U and hence simple. Terminal-rooted subtrees are simple as well.

2. The repeated label must include the root. Let the root label of U be B. Because U is non-simple, some path has a repeated nonterminal. If both occurrences were strictly below the root, they would lie in one child subtree of U, contradicting simplicity of that subtree. Thus B occurs again at a proper descendant production node. Choose one such occurrence, and let V be its complete B-rooted subtree.

3. Extract the context. Replace this occurrence of V in U by $\Box_B$; call the resulting open tree C. It has exactly one hole. Its root and hole label are both B. Since the chosen occurrence was a proper descendant, the original root production remains, so C is nontrivial. All unchanged productions remain valid, and the replacement hole has the required root symbol. Therefore C is a cycle context and $U = C[V]$.

4. Bound its height. Each child subtree of U is simple and therefore has height at most n. Hence $\operatorname{ht}(U) \le n + 1$. Replacing a subtree by a height-zero hole cannot increase height, so $\operatorname{ht}(C) \le n + 1$. Thus $C \in \mathcal{C}$.

5. Cut it out of the original tree. Let E be the outer context selecting the occurrence of U in T. Then

$$T = E[U] = E[C[V]].$$

Define $T' = E[V]$. Since U and V have the same root label, T' is a complete valid A-tree. Inserting C at the occurrence of V gives T exactly. The size identities give

$$\operatorname{size}(T) = \operatorname{size}(E) + \operatorname{size}(C) + \operatorname{size}(V) = \operatorname{size}(T') + \operatorname{size}(C) > \operatorname{size}(T'),$$

because $\operatorname{size}(C) \ge 1$. □

> **Key point.** The decrease is in production-node count, not terminal yield length. A removable context may produce no terminal letters at all.

5.3 Decomposing every complete tree

Theorem 5.6 (Exact decomposition)**.** *For every complete A-rooted tree T, there exist a simple complete A-rooted tree B and a finite insertion sequence*

$$B = T_0 \xrightarrow{C_1} T_1 \xrightarrow{C_2} \cdots \xrightarrow{C_m} T_m = T \tag{5.1}$$

with $C_i \in \mathcal{C}$ for every i. If $A = S$, then $B \in \mathcal{B}$.

Proof. Use strong induction on size(T), with the statement quantified over all root nonterminals and all trees of that size.

If T is simple, take $B = T$ and the empty sequence. Otherwise, lemma 5.5 gives T' and $C \in \mathcal{C}$ with size(T') < size(T) and $T' \xrightarrow{C} T$. Apply the induction hypothesis to T'. It supplies a simple tree B of the same root and a sequence from B to T' using contexts in $\mathcal{C}$. Append the single insertion $T' \xrightarrow{C} T$. The resulting sequence has all required properties. □

There is no uniqueness claim. Different minimal subtrees or repeated occurrences can lead to different decompositions. Existence is enough. If a deterministic algorithm is wanted, fix orders on productions and child positions and always take the first eligible choice.

Corollary 5.7 (Exact generation by the finite families)**.** *The complete S-rooted trees are exactly the trees obtainable from some $B \in \mathcal{B}$ by a finite valid sequence of insertions of contexts from $\mathcal{C}$.*

Proof. The forward inclusion is theorem 5.6. For the reverse inclusion, a base is a complete S-tree and every insertion preserves completeness, grammar validity, and root symbol by lemma 4.5. □

6 Which sets of contexts can be used together?

Fix a complete nonterminal-rooted tree B and a finite set D of cycle contexts. For this section, B need not be simple and the contexts need not have bounded height. This separates the activation argument from the particular bounds of the preceding section.

6.1 An explicit compatibility condition

Definition 6.1 (Activatable set)**.** The set D is *activatable from* B, written Act(B, D), if its elements admit an enumeration without repetition $C_1, \ldots, C_k$ such that, for $1 \le i \le k$,

$$\mathrm{root}(C_i) \in \mathrm{supp}(B) \cup \bigcup_{j<i} \mathrm{supp}(C_j). \tag{6.1}$$

The empty set is activatable using the empty enumeration.

This is a finite condition on labels. It says that each context can be introduced after the preceding ones. A context cannot justify its own first insertion merely because its root label occurs inside itself.

Lemma 6.2 (An activation order can be realized)**.** *If $C_1, \ldots, C_k$ witnesses* Act(B, D)*, then there is a valid insertion sequence from B using each of these contexts exactly once, in that order. The resulting tree B_D has*

$$\mathrm{wt}(B_D) = \mathrm{wt}(B) + \sum_{C \in D} \mathrm{wt}(C), \tag{6.2}$$

$$\mathrm{supp}(B_D) = \mathrm{supp}(B) \cup \bigcup_{C \in D} \mathrm{supp}(C). \tag{6.3}$$

In particular, every $C \in D$ is enabled in B_D.

Proof. Construct the sequence inductively. Before step i, the support accounting identity is

$$\operatorname{supp}(B) \cup \bigcup_{j<i} \operatorname{supp}(C_j).$$

By the activation condition it contains $\operatorname{root}(C_i)$. The enabledness lemma therefore supplies the next insertion. The insertion laws update both weight and support, maintaining the induction invariant. This proves the two final identities.

Finally, $\operatorname{root}(C) \in \operatorname{supp}(C)$ for every cycle context C, so the final support contains every required root. By lemma 4.4, every member of D is enabled there. □

6.2 Why first occurrences suffice

Lemma 6.3 (First-occurrence lemma)**.** *Suppose a finite valid insertion sequence starts at B, and let D be the set of distinct contexts occurring in it. Then* $\operatorname{Act}(B, D)$.

Proof. Write the sequence labels as $E_1, \ldots, E_m$. List their distinct values in order of first occurrence as $C_1, \ldots, C_k$. Let j_i be the index of the first occurrence of C_i.

The insertion of $E_{j_i} = C_i$ was valid, so its root is present immediately before that step. By corollary 4.6,

$$\operatorname{root}(C_i) \in \operatorname{supp}(B) \cup \bigcup_{\ell<j_i} \operatorname{supp}(E_\ell).$$

The distinct contexts occurring before index j_i are exactly $C_1, \ldots, C_{i-1}$. Repeating a set in a union does not change the union. Thus

$$\bigcup_{\ell<j_i} \operatorname{supp}(E_\ell) = \bigcup_{r<i} \operatorname{supp}(C_r).$$

Substituting gives the activation condition for C_i. This holds for every i. □

The proof does not remove repeated contexts from the final tree in place. Instead, it extracts a valid order from support information and then uses lemma 6.2 to build a new tree. This distinction avoids having to preserve particular insertion positions after deletions.

6.3 After activation, multiplicities are independent

Lemma 6.4 (Arbitrary positive multiplicities)**.** *Assume* $\operatorname{Act}(B, D)$. *For every family $(m_C)_{C \in D}$ of positive integers, there is a valid insertion sequence from B in which each $C \in D$ occurs exactly m_C times and no other context occurs.*

Proof. First use lemma 6.2 to insert each member of D once. Every member of D is now enabled. For each $C \in D$, perform $m_C - 1$ additional insertions of C, choosing any occurrence of its root label each time.

Every extra insertion is possible: supports only grow under insertion, so all roots that were enabled after the first pass remain present. There are finitely many requested insertions because D is finite and each m_C is finite. The resulting multiplicities are exactly the requested ones. □

Definition 6.5 (Trees using exactly a given set of contexts)**.** Let $\mathcal{T}(B, D)$ be the set of complete trees obtainable from B by a finite insertion sequence whose set of context labels is exactly D. Thus every member of D must occur at least once, and no context outside D may occur.

Proposition 6.6 (One activatable set gives one linear set)**.** *If* $\mathrm{Act}(B, D)$*, then*

$$\{\mathrm{wt}(T) : T \in \mathcal{T}(B, D)\} = \mathrm{Lin}\left(\mathrm{wt}(B) + \sum_{C \in D} \mathrm{wt}(C); (\mathrm{wt}(C))_{C \in D}\right). \tag{6.4}$$

If $\mathrm{Act}(B, D)$ *fails, then* $\mathcal{T}(B, D) = \varnothing$*.*

Proof. Suppose first that $\mathrm{Act}(B, D)$ holds.

For the left-to-right inclusion, choose a sequence witnessing $T \in \mathcal{T}(B, D)$. Let $m_C \geq 1$ count the occurrences of C. Weight accounting gives

$$\mathrm{wt}(T) = \mathrm{wt}(B) + \sum_{C \in D} m_C \,\mathrm{wt}(C) = \left(\mathrm{wt}(B) + \sum_{C \in D} \mathrm{wt}(C)\right) + \sum_{C \in D} (m_C - 1)\,\mathrm{wt}(C).$$

Since every $m_C - 1$ is a natural number, this vector belongs to the displayed linear set.

For the right-to-left inclusion, choose any coefficients $k_C \in \mathbb{N}$. By lemma 6.4, there is a sequence using C exactly $k_C + 1$ times. Its final weight is the desired base plus $\sum_{C \in D} k_C \,\mathrm{wt}(C)$.

Finally, if $\mathcal{T}(B, D)$ were nonempty, a witnessing sequence would imply $\mathrm{Act}(B, D)$ by lemma 6.3. The contrapositive proves the last assertion. □

> **Key point.** Support determines whether a set of contexts can be used. Positive multiplicities determine how much each activated context contributes. These are separate pieces of information.

7 The exact semilinear representation

Return to the families $\mathcal{B}$ and $\mathcal{C}$ in definition 5.3. For each $B \in \mathcal{B}$ and $D \subseteq \mathcal{C}$ with $\mathrm{Act}(B, D)$, define

$$b_{B,D} = \mathrm{wt}(B) + \sum_{C \in D} \mathrm{wt}(C), \tag{7.1}$$

$$X_{B,D} = \mathrm{Lin}\big(b_{B,D}; (\mathrm{wt}(C))_{C \in D}\big). \tag{7.2}$$

Theorem 7.1 (Explicit equality)**.** *For every finite context-free grammar* G*,*

$$\boxed{\Psi(L(G)) = \bigcup_{B \in \mathcal{B}} \bigcup_{\substack{D \subseteq \mathcal{C} \\ \mathrm{Act}(B,D)}} X_{B,D}.} \tag{7.3}$$

In particular, $\Psi(L(G))$ *is semilinear.*

Proof. **First inclusion: every generated word is represented.** Take $x \in \Psi(L(G))$. There is a word $w \in L(G)$ with $x = \Psi(w)$. By proposition 3.3, choose a complete S-tree T with $\mathrm{yield}(T) = w$. Then $x = \mathrm{wt}(T)$.

By theorem 5.6, there is a base $B \in \mathcal{B}$ and an insertion sequence from B to T, all of whose contexts belong to $\mathcal{C}$. Let D be its set of distinct contexts. Then $D \subseteq \mathcal{C}$, and lemma 6.3 gives $\mathrm{Act}(B, D)$. By proposition 6.6, $\mathrm{wt}(T) \in X_{B,D}$. Hence x belongs to the right-hand side of (7.3).

Second inclusion: every represented vector is generated. Take x from the right-hand side. It lies in some $X_{B,D}$ with $B \in \mathcal{B}$ and $\mathrm{Act}(B, D)$. By proposition 6.6, there is a tree $T \in \mathcal{T}(B, D)$ with weight x. Since B is a complete S-tree and all insertions preserve grammar validity and root symbol, so is T. Therefore $\mathrm{yield}(T) \in L(G)$ and $x = \Psi(\mathrm{yield}(T)) \in \Psi(L(G))$.

Finiteness. By lemma 5.4, $\mathcal{B}$ and $\mathcal{C}$ are finite. The power set of $\mathcal{C}$ is finite, so the displayed union is finite. Each $X_{B,D}$ is linear by its definition. This proves semilinearity. □

This proves an equality, not merely a semilinear overapproximation. The second inclusion is what prevents the construction from accepting vectors obtained by adding incompatible periods.

7.1 What is and is not canonical

The same vector can lie in several components, and distinct components can even denote the same set. A context with zero weight gives a zero period. Distinct contexts can give identical nonzero periods. None of this affects correctness.

The proof retains contexts as indices until after activation has been checked. One should not identify two contexts merely because their weights agree: their roots and supports can differ, so their roles in activation can differ as well. Within an already constructed linear component, duplicate or zero period vectors may be removed if desired.

8 A worked example: activation has a cost

Consider the grammar over the ordered alphabet (a, b, c) with productions

$$S \to a \mid AS, \qquad A \to b \mid cA.$$

A word is a sequence of zero or more blocks $c^j b$, followed by a final a. Thus

$$L(G) = (c^* b)^* a.$$

The empty sequence of blocks gives a. With at least one block, any number of c's can occur. We therefore expect

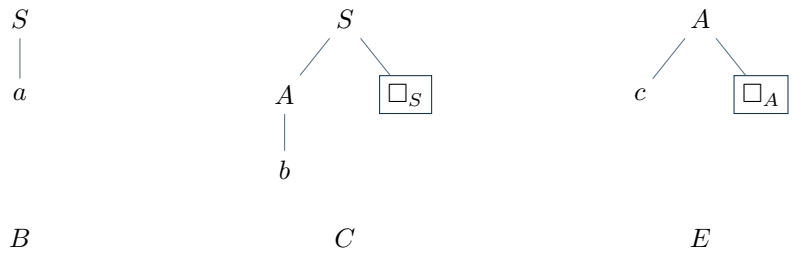
$$\Psi(L(G)) = \{(1, 0, 0)\} \ \cup \ \{(1, k, \ell) : k \geq 1, \ \ell \geq 0\}. \tag{8.1}$$

The example has one simple S-tree B, namely the tree for $S \to a$. Consider these two cycle contexts:

$$C = S(A(b), \square_S), \qquad E = A(c, \square_A).$$

Here and in the drawing, node labels abbreviate their uniquely determined productions. The relevant data are

Object	Root	Support	Weight
B	S	$\{S\}$	$(1, 0, 0)$
C	S	$\{S, A\}$	$(0, 1, 0)$
E	A	$\{A\}$	$(0, 0, 1)$



Initially E is not enabled: B contains no A-node. Inserting C once yields the tree for ba, and introduces an A-node. Context E is now enabled and can be inserted there any number of times. Further copies of C can also be inserted at any S-node.

The family $\{C, E\}$ suffices to generate all complete S-trees for this example. Each use of $S \to AS$ can be built by one insertion of C; the new A-branch initially uses $A \to b$. Its uses of $A \to cA$ can then be added by insertions of E. This small family is enough for the calculation below; the general bounded family $\mathcal{C}$ may contain additional, redundant contexts.

Chosen set	Activatable?	Resulting linear component
$\varnothing$	Yes	$\{(1,0,0)\}$
$\{C\}$	Yes	$\{(1,1,0) + m(0,1,0) : m \in \mathbb{N}\}$
$\{E\}$	No	None: the root A is absent from B
$\{C, E\}$	Yes, C then E	$\{(1,1,1) + m(0,1,0) + r(0,0,1) : m, r \in \mathbb{N}\}$

The union of the three valid components is exactly (8.1). The two nonsingleton components can also be merged to obtain

$$\{(1,0,0)\} \ \cup \ \mathrm{Lin}\big((1,1,0); (0,1,0), (0,0,1)\big).$$

This simplification is optional; the proof does not require a minimal representation.

Why the base shift matters. If we incorrectly used base $\mathrm{wt}(B) = (1,0,0)$ with periods $\mathrm{wt}(C)$ and $\mathrm{wt}(E)$, choosing zero copies of C and one copy of E would yield $(1,0,1)$. No generated word has this vector: every c belongs to a block ending in b. In the correct construction, using E requires first activating C, whose compulsory contribution places a b in the base vector.

9 The converse: a regular language for every semilinear set

Choose an order $a_1, \ldots, a_d$ on Σ. For $v \in \mathbb{N}^\Sigma$, define a representative word

$$\mathrm{word}(v) = a_1^{v(a_1)} \cdots a_d^{v(a_d)}.$$

By construction,

$$\Psi(\mathrm{word}(v)) = v. \tag{9.1}$$

When Σ is empty, the product is empty and the representative word is ε.

Lemma 9.1 (A regular expression for one linear set)**.** *Let* $X = \mathrm{Lin}(b; p_1, \ldots, p_k)$. *Set* $u = \mathrm{word}(b)$ *and* $v_i = \mathrm{word}(p_i)$. *The regular language*

$$R_X = u\, v_1^* \cdots v_k^*$$

satisfies $\Psi(R_X) = X$.

Proof. If $w \in R_X$, then for some $m_1, \dots, m_k \in \mathbb{N}$,

$$w = u v_1^{m_1} \cdots v_k^{m_k}.$$

By additivity and (9.1),

$$\Psi(w) = b + \sum_{i=1}^{k} m_i p_i \in X.$$

Conversely, given $x = b + \sum_i m_i p_i \in X$, the displayed word belongs to R_X and has Parikh vector x. Thus both inclusions hold. The expression uses singleton words, concatenation, and Kleene star, so it denotes a regular language. □

Theorem 9.2 (Semilinear sets are regular Parikh images)**.** *If*

$$X = \bigcup_{j=1}^{r} \mathrm{Lin}(b_j; p_{j,1}, \dots, p_{j,k_j}),$$

then the regular language

$$R = \bigcup_{j=1}^{r} \mathrm{word}(b_j)\ \mathrm{word}(p_{j,1})^* \cdots \mathrm{word}(p_{j,k_j})^* \tag{9.2}$$

satisfies $\Psi(R) = X$.

Proof. For any finite collection of languages $(L_j)_j$, membership shows that $\Psi(\bigcup_j L_j) = \bigcup_j \Psi(L_j)$: a word belongs to the union precisely when it belongs to one member. Apply lemma 9.1 to each component. Finite union preserves regularity. □

For $r = 0$, choose $R = \varnothing$. For $k_j = 0$, the corresponding component is the singleton word $\mathrm{word}(b_j)$. A zero period gives $\varepsilon^* = \{\varepsilon\}$ and causes no difficulty.

Combining theorems 7.1 and 9.2 proves both directions requested in theorem 1.4. In particular, applying (9.2) to the explicit components in (7.3) yields a regular language with exactly the same Parikh image as the given CFL.

Remark 9.3 (The regular language need not be a sublanguage). The representative word for a vector orders all copies of a_1 before all copies of a_2, and so on. That word need not belong to the original CFL. The construction preserves terminal counts; it is not a construction of a regular sublanguage with those counts.

10 Effectivity and an explicit grammar-size bound

10.1 Deciding activation by a finite procedure

For a proposed pair (B, D), maintain a set U of available nonterminal labels and a set F of contexts already processed. Initially

$$U = \mathrm{supp}(B), \qquad F = \varnothing.$$

Repeatedly select a context $C \in D \setminus F$ with $\mathrm{root}(C) \in U$, and update

$$U \leftarrow U \cup \mathrm{supp}(C), \qquad F \leftarrow F \cup \{C\}.$$

Accept if $F = D$. Reject if $F \neq D$ and no selection is possible.

Lemma 10.1 (Correctness of the activation test)**.** *This procedure terminates after at most $|D|$ selections, and accepts exactly when* $\mathrm{Act}(B, D)$ *holds. Any choice among currently enabled unprocessed contexts is safe.*

Proof. Each selection increases $|F|$ by one, so at most $|D|$ selections occur. The invariant is

$$U = \mathrm{supp}(B) \cup \bigcup_{C \in F} \mathrm{supp}(C).$$

If the procedure accepts, the selection order is an activation order by the condition used at each step and this invariant.

Conversely, suppose an activation order exists and the procedure stops with some contexts unprocessed. Take the first unprocessed context in that activation order. Every context preceding it in the order is processed. Its required root therefore belongs to the current U, by the activation condition and the invariant. It is an enabled unprocessed context, contradicting the reason for rejection. Thus rejection is impossible when an activation order exists. This reasoning did not constrain which eligible context was selected at earlier steps. □

10.2 The algorithm producing the representation

The following is mathematical pseudocode. Every enumeration is over a proved finite domain.

```
semilinear_representation(G):
    n := number of nonterminals
    B := all simple complete S-trees of height at most n
    C := all nontrivial same-root/hole contexts
         of height at most n + 1
    result := empty list
    for each base tree b in B:
        for each subset D of C:
            if activation_test(b, D) accepts:
                base := weight(b) + sum(weight(c) for c in D)
                periods := [weight(c) for c in D]
                append (base, periods) to result
    return result
```

The bounded-tree enumeration is described in lemma 5.4; the activation test is correct by lemma 10.1; and the resulting list denotes exactly $\Psi(L(G))$ by theorem 7.1. No test asks whether an arbitrary word belongs to the grammar or whether an unbounded derivation exists.

10.3 A fully explicit, deliberately coarse bound

We now quantify the phrase "a function of the grammar." Let

$$n = |N|, \qquad d = |\Sigma|, \qquad p = |P|, \qquad b = \max(\{1\} \cup \{|\alpha| : (A \to \alpha) \in P\}).$$

Define

$$M = \sum_{i=0}^{n+1} b^i, \qquad q = 1 + d + n + p, \qquad K = q^M. \tag{10.1}$$

These quantities are determined by the finite grammar description. Their purpose is to give a transparent upper bound, not an efficient one.

Lemma 10.2 (Counting bounded trees)**.** *Both $|\mathcal{B}|$ and $|\mathcal{C}|$ are at most K. Every $B \in \mathcal{B}$ and $C \in \mathcal{C}$ has at most M terminal leaves, and hence*

$$\|\operatorname{wt}(B)\|_1 \le M, \qquad \|\operatorname{wt}(C)\|_1 \le M,$$

where $\|v\|_1 = \sum_{a\in\Sigma} v(a)$.

Proof. Every production has at most b children. A tree whose production height is at most $n+1$ has ordinary edge depth at most $n+1$: all ancestors of a terminal leaf or hole are production nodes, and a path ending at a production node has no greater edge depth than its production count. Consequently every node can be assigned a position in the full ordered b-ary tree through depth $n+1$. This position set has M elements.

At each position, record either absence, one of d terminal labels, one of n hole labels, or one of p production labels. There are q possible records. This encodes every bounded valid open grammar tree injectively into the $q^M = K$ records on the position set. Most records do not encode valid trees, but that only makes this an overcount. Complete simple trees and bounded cycle contexts are both among the encoded trees.

The same position bound gives at most M nodes in each tree, in particular at most M terminal leaves. Each terminal leaf contributes one to the ℓ_1 norm of its weight, while production nodes and holes contribute zero. □

Theorem 10.3 (Size bound for the produced semilinear representation)**.** *With the quantities in (10.1), the construction produces a representation with*

(i) at most $K\,2^K$ linear components;
(ii) at most K period vectors per component;
(iii) every period vector having ℓ_1 norm at most M;
(iv) every base vector having ℓ_1 norm at most $(K+1)M$.

These are bounds on one representation guaranteed to exist, not lower bounds or optimal bounds.

Proof. There are at most $|\mathcal{B}|2^{|\mathcal{C}|} \le K2^K$ candidate pairs (B, D); activation filtering can only reduce this number. A component indexed by (B, D) has $|D| \le |\mathcal{C}| \le K$ indexed periods. Each period is the weight of a single bounded context and is bounded by lemma 10.2.

Finally, the base is a sum of one complete-tree weight and $|D|$ context weights. Nonnegative coordinates imply additivity of ℓ_1 norm. Therefore

$$\|b_{B,D}\|_1 \le M + |D|M \le (K+1)M.$$

□

Binary encoding size. Set

$$L = \max\left(1, \left\lceil \log_2\left((K+1)M+1\right)\right\rceil\right).$$

Every coordinate of every base or period fits in L bits. Thus the numerical vector entries occupy at most

$$K2^K\,(K+1)\,d\,L \tag{10.2}$$

bits in a format with fixed-width coordinates, excluding list-length metadata. The metadata is also bounded in terms of K: one count for the components and one count, at most K, for the

periods in each component. For example, storing these counts with widths $\lceil \log_2(K2^K + 1) \rceil$ and $\lceil \log_2(K + 1) \rceil$ adds at most

$$\left\lceil \log_2(K2^K + 1) \right\rceil + K2^K \lceil \log_2(K + 1) \rceil$$

bits. The alphabet order and these fixed widths are treated as known from G.

If one prefers a single scalar grammar-size parameter, take

$$g = 1 + n + d + p + \sum_{(A \to \alpha) \in P} |\alpha|.$$

Then $n, d, p, b \le g$ and $q \le g$. In particular, replacing M by $\widehat{M} = \sum_{i=0}^{g+1} g^i$ and K by $\widehat{K} = g^{\widehat{M}}$ gives bounds depending on g alone. Here $g \ge 2$, since $n \ge 1$.

> **Key point.** The proof gives a finite representation whose size depends only on the grammar, uniformly over all words it generates. No bound depends on the length of an individual word.

10.4 Edge cases that the proof already covers

The empty language.
If no complete S-tree exists, then $\mathcal{B} = \varnothing$, and the union is empty. Conversely, any complete S-tree would decompose to a member of $\mathcal{B}$, so this case is detected by the finite construction.

The empty word.
Trees using empty productions are ordinary complete trees in our representation. Their weight can be zero. The final equality includes 0 exactly when ε is generated, since a word has zero counts at every terminal precisely when it is empty.

Unit and silent cycles.
A cycle such as $A \to B \to A$ can give a context of weight zero. It still has positive production-node size, so cutting terminates. Zero periods are legal.

Unreachable or unproductive nonterminals.
They do not invalidate bounded enumeration. Only valid complete bases are used, and every activated insertion yields a complete valid tree. Impossible uses cannot create vectors in the final union.

Ambiguity.
Multiple trees or insertion sequences may witness the same vector. Both sides of the theorem are sets, so uniqueness is unnecessary.

The empty terminal alphabet.
The space $\mathbb{N}^{\varnothing}$ has one vector, 0. A generated language is either empty or $\{\varepsilon\}$. All definitions and both constructions still apply.

11 A blueprint for mechanization

This section describes an implementation plan for Lean or another proof assistant. The declarations below are specifications of mathematical interfaces, not executable library code. No claim is made about existing theorem names or the current contents of any library.

11.1 Start with witness data, not just an existential theorem

A semilinear witness can be a finite list of components. Each component contains a base vector and a finite list of period vectors. Define its denotation by

$$[\![(b, [p_1, \ldots, p_k])]\!] = \left\{ b + \sum_{i=1}^{k} m_i p_i : m_i \in \mathbb{N} \right\},$$

and define the denotation of a list of components as the union of their denotations. Then the target theorem has the form

$$[\![\text{representation}(G)]\!] = \Psi(L(G)).$$

This makes the effective content concrete. The existence theorem follows immediately by taking the constructed list as witness. Size bounds become statements about its length, period-list lengths, and coordinates.

For a finite alphabet, vectors can be functions $\Sigma \to \mathbb{N}$. Prove vector equalities by function extensionality when needed. Finite sums handle weights and periods uniformly. There is no mathematical need to introduce integer vectors or subtraction of vectors.

11.2 Represent validity explicitly

A convenient raw syntax is

```
RawTree := terminal(a)
         | hole(A)
         | production(p, children)
```

where $a \in \Sigma$, $A \in N$, $p \in P$, and children is a finite ordered list of raw trees. A validity predicate checks that the child-root symbols equal the right-hand side of p, including its length. This separates finite syntax from grammar correctness.

The main tree predicates are then:

- `Complete(T)`: T is valid and has zero holes;
- `OneHole(E,A)`: E is valid and has exactly one hole, labelled A;
- `Cycle(C,A)`: C is a nontrivial one-hole context with root A and hole A;
- `Simple(T)`: no path repeats a production-node nonterminal.

The types $\mathcal{T}_A$ and $\mathcal{C}$ can be subtypes carrying these predicates. Alternatively, use intrinsically typed trees; the mathematical proof does not depend on that implementation choice.

For positions, use lists of natural numbers. A replacement operation should require a valid position and preserve the root symbol at that position. A zipper representation for one-hole contexts is another possible implementation. Choose one replacement interface and prove its laws once.

11.3 The modules and their contracts

Module	Main contract	Mathematical reference
Word counting	Counts of concatenation add; representative words have the prescribed counts	(1.1), (9.1)
Tree statistics	Recursive weight, support, production count, and production height	definition 3.2
Plugging	Preserves validity and root; adds weight and size; unions support	lemma 4.2
Insertion	Enabled exactly by root membership in support; preserves old support	lemmas 4.4 and 4.5
Bounded syntax	Finite, enumerable trees of bounded height; finite bases and contexts	lemma 5.4
Bounded cut	A non-simple tree is one bounded-context insertion into a smaller tree	lemma 5.5
Decomposition	Exact finite reconstruction from a simple base	theorem 5.6
Activation	First occurrences form a valid activation order	lemma 6.3
Multiplicity	Every positive multiplicity assignment can be realized	lemma 6.4
Linear component	Exact weight set for an activatable context set	proposition 6.6
Global equality	Union of all produced components equals the grammar's Parikh image	theorem 7.1
Quantitative bound	Component count, periods, and vector norms depend only on G	theorem 10.3
Regular realization	Build an expression for a finite semilinear witness	theorem 9.2

11.4 Induction measures and finite choices

There are three distinct inductions. Keeping them separate makes termination arguments local.

1. **Structural induction on a context or its hole path** proves plugging and replacement laws. Child-list sums and unions are the main algebraic bookkeeping.
2. **Strong induction on production-node size** proves exact decomposition. The bounded-cut lemma supplies the strict decrease. Terminal count is unsuitable because a cut can have weight zero.
3. **Induction on sequence length** proves weight/support accounting and realizes an activation order. After activation, induction on the number of remaining requested insertions realizes multiplicities.

All choices in the constructive version range over finite sets: a minimal non-simple subtree, a repeated descendant label, an occurrence of a root label, or a currently enabled context. Finite enumerations with decidable equality can supply these choices by search. The proof therefore needs no oracle for unbounded grammar behavior.

For the minimal-subtree step, one implementation scans positions, retains positions whose subtree is non-simple, and chooses a minimum by production-node size. A descendant proper

subtree has strictly fewer production nodes because the parent production is lost. The selected subtree consequently has no non-simple proper subtree.

11.5 The finite-support lemma deserves its own interface

The activation proof can be isolated from grammar syntax. Its abstract data are a state support $U \subseteq N$ and operations C, each with a required label $\mathrm{root}(C)$ and an added finite set $\mathrm{supp}(C)$. An operation is allowed if its required label is present, and changes support to $U \cup \mathrm{supp}(C)$.

For cycle contexts, additionally $\mathrm{root}(C) \in \mathrm{supp}(C)$. The concrete insertion lemmas establish the link between this support transition and actual trees. Once that link is proved, the first-occurrence argument is a theorem about finite sets and lists.

This separation gives a useful debugging discipline: if arbitrary positive multiplicities are difficult to prove at the tree level, first prove the support version. Then lift each enabled support step to an actual insertion using lemma 4.4. No fixed insertion position has to survive the entire construction.

11.6 A recommended order of milestones

1. Define complete trees and contexts, and prove the three plugging identities.
2. Define insertion as a relation and prove its enabledness and accounting laws.
3. Prove the bounded-cut lemma and exact decomposition, initially without quantitative enumeration bounds.
4. Prove activation and positive-multiplicity realization for an arbitrary finite family of cycle contexts.
5. Assemble the forward equality using finite bases and contexts.
6. Define representative words and prove the converse independently.
7. Add executable bounded enumeration, the activation test, and the numerical size bounds.
8. Connect tree semantics to the chosen library's definition of context-free language, using Appendix A if needed.

This ordering allows the core mathematical argument to be checked before committing to the representation of finite enumerations. If a verified CFG-to-tree correspondence is already available, the final step can instead be an early adapter theorem.

11.7 Common gaps to guard against

Finding one pump is not a semilinear representation.
The proof needs a finite grammar-dependent family of pumps covering every tree. The bounded-cut and decomposition lemmas provide that family.

Bounded height needs bounded branching.
The grammar is finite, so all right-hand sides have a common finite arity bound. Without this fact, a height bound alone would not imply finitely many ordered trees.

The base must account for first occurrences.
If a context is in D, its multiplicity in $\mathcal{T}(B, D)$ is at least one. The shift $\sum_{C \in D} \mathrm{wt}(C)$ cannot simply be omitted.

Support does not mean a frontier nonterminal.
Complete trees have no unexpanded nonterminal frontier. Insertions occur at internal production nodes, including production nodes with empty right-hand sides.

A context is not just its vector.
Its root and support determine compatibility. Preserve context identities until the linear component is constructed.

The converse preserves an image, not all words with that image.
The expression in (9.2) chooses particular orders of letters. It does not accept every permutation of every vector in the semilinear set.

A Derivation trees and ordinary word derivations

We prove proposition 3.3 without imposing a derivation order or a grammar normal form.

A.1 From a complete tree to a word derivation

First observe the context property of derivations:

$$\alpha \Rightarrow_G^* \beta \quad \Longrightarrow \quad u\alpha v \Rightarrow_G^* u\beta v \tag{A.1}$$

for arbitrary sentential forms u, v. For a single derivation step, replacing a nonterminal in α also replaces the same occurrence inside $u\alpha v$. Induction on the number of steps proves (A.1), including the zero-step case.

We now prove by structural induction on a complete tree T that its root symbol derives its yield. For a terminal leaf a, the derivation is the zero-step derivation $a \Rightarrow_G^* a$.

For $T = \mathrm{node}(p; T_1, \ldots, T_m)$, write $p = (A \to X_1 \cdots X_m)$. Apply p once:

$$A \Rightarrow_G X_1 \cdots X_m.$$

The induction hypotheses give $X_i \Rightarrow_G^* \mathrm{yield}(T_i)$. Use (A.1) to replace X_1 by its yield, then X_2, and so on. Composing these derivations gives

$$A \Rightarrow_G^* \mathrm{yield}(T_1) \cdots \mathrm{yield}(T_m) = \mathrm{yield}(T).$$

For $m = 0$, the production itself is $A \to \varepsilon$, and there are no child derivations to perform. This proves the tree-to-word direction for every root nonterminal.

A.2 From a word derivation to a complete tree

Use valid open grammar trees with any finite number of holes. Define their frontier over $N \uplus \Sigma$ recursively by

$$\begin{aligned} \mathrm{front}(\mathrm{leaf}(a)) &= a, \\ \mathrm{front}(\square_A) &= A, \\ \mathrm{front}(\mathrm{node}(p; T_1, \ldots, T_m)) &= \mathrm{front}(T_1) \cdots \mathrm{front}(T_m). \end{aligned}$$

Production-node labels do not themselves contribute to the frontier. In particular, an empty production contributes the empty word. The nonterminal occurrences in the frontier are exactly the holes, in their left-to-right order.

Suppose $A \Rightarrow_G^* w$. Fix a finite derivation

$$A = \alpha_0 \Rightarrow_G \alpha_1 \Rightarrow_G \cdots \Rightarrow_G \alpha_t = w.$$

We construct valid open trees T_i with root A and frontier α_i.

Start with $T_0 = \square_A$. For step i, the derivation expands one specified nonterminal occurrence, say B, using $p = (B \to X_1 \cdots X_m)$. Since nonterminal frontier occurrences correspond exactly to holes, there is a corresponding hole $\square_B$ in T_i. Replace that hole by the production node for p. Give it a terminal leaf for every terminal X_j, and a hole $\square_{X_j}$ for every nonterminal X_j. These children have precisely the required root symbols, so the tree remains valid. Its frontier is exactly α_{i+1}, and its root symbol remains A.

At the end, $\mathrm{front}(T_t) = w$ contains only terminals. Every hole would contribute a nonterminal to the frontier, so T_t has no holes. It is therefore a complete A-tree. For complete trees, the frontier agrees with the terminal yield, so $\mathrm{yield}(T_t) = w$.

This proves the word-to-tree direction and finishes the proof of proposition 3.3. Notice that the construction works for any order of grammar-rule applications; no leftmost-derivation theorem is required.

B An immediate arithmetic corollary

The semilinear representation gives an explicit existential formula in Presburger arithmetic. This is a consequence of the proved theorem, not a prerequisite for it.

For one component $X = \mathrm{Lin}(b; p_1, \ldots, p_k)$, define

$$\varphi_X(x_1, \ldots, x_d) \equiv \exists m_1, \ldots, m_k \in \mathbb{N} \bigwedge_{j=1}^{d} \left(x_j = b_j + \sum_{i=1}^{k} m_i (p_i)_j \right).$$

The numbers $(p_i)_j$ are fixed constants. Multiplication by one of these constants is shorthand for repeated addition, so this is a Presburger formula. By the definition of a linear set, it holds exactly for the vectors in X.

For a finite union, take the disjunction of the component formulas. The empty union gives false. Thus the construction also gives a Presburger definition of $\Psi(L(G))$.

The reverse theorem, that every Presburger-definable subset of $\mathbb{N}^d$ is semilinear, is a separate result and is not proved or used here. Both directions of the language-theoretic theorem requested in this note have already been proved in Sections 7 and 9.

C Notation and the final witness at a glance

Notation	Meaning
$\Psi(w)$	Terminal-count vector of a word
$\mathcal{T}_A$	Complete grammar trees rooted at A
$\mathrm{wt}(T)$	Terminal-count vector of a tree or context
$\mathrm{size}(T)$	Number of production nodes
$\mathrm{ht}(T)$	Maximum number of production nodes along a path
$\mathrm{supp}(T)$	Nonterminal labels on production nodes; holes excluded
$E[T]$	Plug a matching complete tree into a one-hole context
$T \xrightarrow{C} T'$	Insert a cycle context around one matching subtree
$\mathcal{B}$	Simple complete S-trees
$\mathcal{C}$	Nontrivial cycle contexts of height at most $\lvert N\rvert + 1$
$\mathrm{Act}(B, D)$	D has an order in which each required root is already present
$b_{B,D}$	$\mathrm{wt}(B) + \sum_{C \in D} \mathrm{wt}(C)$
$X_{B,D}$	Linear set with base $b_{B,D}$ and periods $(\mathrm{wt}(C))_{C \in D}$

The complete witness is the finite list

$$\left[\left(\mathrm{wt}(B) + \sum_{C \in D} \mathrm{wt}(C),\ [\mathrm{wt}(C) : C \in D]\right)\ :\ B \in \mathcal{B},\ D \subseteq \mathcal{C},\ \mathrm{Act}(B, D)\right].$$

Its denotation is exactly $\Psi(L(G))$. The list has at most $K2^K$ components, at most K periods per component, period norms at most M, and base norms at most $(K+1)M$, with M, K defined in (10.1).

Sources and relation to other proofs

Parikh's original theorem is due to Rohit Parikh [1]. The proof developed here belongs to the derivation-tree decomposition approach: bounded complete trees provide initial objects, and one-hole contexts provide repeatable additions. Short presentations of this approach include Sin'ya's account of the Takahashi method [2] and Rubtsov's derivation-tree proof [3].

This note uses a deliberately generous height-bounded context family, explicit support-based activation, and a coarse syntax-counting bound. The aim is a self-contained sequence of proof obligations suitable for mechanization, rather than a short proof or a tight complexity bound. Every mathematical lemma needed for the two stated language-theoretic directions is proved above.

For a different route, Esparza, Ganty, Kiefer, and Luttenberger give a direct automaton construction [4]. An existing Isabelle formalization by Fabian Lehr follows Pilling's approach through systems of language equations [5]. That development is evidence of a different formalization strategy; it is not a dependency of this note.

References

[1] Rohit J. Parikh. *On context-free languages*. Journal of the ACM, 13(4):570–581, 1966. doi:10.1145/321356.321364.

[2] Ryoma Sin'ya. *Simple proof of Parikh's theorem à la Takahashi*. 2019. arXiv:1909.09393.

[3] Alexander Rubtsov. *The simplest proof of Parikh's theorem via derivation trees*. 2022. arXiv:2301.00047.

[4] Javier Esparza, Pierre Ganty, Stefan Kiefer, and Michael Luttenberger. *Parikh's theorem: A simple and direct automaton construction*. Information Processing Letters, 111(12):614–619, 2011. doi:10.1016/j.ipl.2011.03.019.

[5] Fabian Lehr. *Parikh's Theorem*. Archive of Formal Proofs, 26 June 2025. `https://isa-afp.org/entries/Parikh.html`.